

Break Statement

The `break` statement is used to immediately terminate the current execution of a loop or a switch statement using the `break` keyword. When it is encountered inside the body or block, program control exits from that loop or switch and resumes execution at the next statement following it.

In looping statements, the `break` keyword is generally used along with an if-condition. Once the specified condition is satisfied, the `break` statement is executed, and the loop is terminated, regardless of whether the loop's original condition is still true. Thus, it allows early exit from the loop based on a specific requirement.

Normally, a looping statement continues execution as long as its given condition remains satisfied and automatically terminates when that condition becomes false. However, if there is a need to skip the remaining part of the loop or to leave the loop before the condition fails, the `break` statement is used.

Similarly, when a `break` statement is executed inside a switch block, it terminates the execution of the current case and prevents the execution of subsequent case statements.

Nested Loop Case

When a `break` statement is encountered within the body of an inner loop, it terminates only the execution of that inner loop. As a result, program control exits from the inner loop and continues with the next iteration of the outer loop.

Syntax

```
break;
```

Structure of break Statement in Looping Statements

1. for Loop

```
for (initialization; condition; increment/decrement)
{
    -----
    if (condition)
        break;
    -----
}
next_statement;
```

2. while Loop

```

while (condition)
{
    -----
    if (condition)
        break;
    -----
}
next_statement;

```

3. do-while Loop

```

do
{
    -----
    if (condition)
        break;
    -----
}
while (condition);
next_statement;

```

Example: Break Statement

```

public class BreakExample {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            if (i == 3) {
                break;    // Stop the loop completely when i is 3
            }
            System.out.println(i);
        }
        System.out.println("Loop ended.");
    }
}

```

Explanation: - The for loop starts with $i = 1$ and runs until $i \leq 5$. - Inside the loop, there is an if statement that checks if i is equal to 3. - When i is 3, the break statement is executed. This terminates the loop immediately, so no further iterations occur. - For $i = 1$ and $i = 2$, the loop prints the value of i . - The loop stops at $i = 3$ and does not print 3, 4, or 5. - After the loop ends, the statement `System.out.println("Loop ended.");` is executed.

Output:

```
1
2
Loop ended.
```

Continue Statement

The `continue` statement is used to stop the execution of the current iteration, thereby preventing the remaining code in that iteration from being executed, and program control moves directly to the next iteration of the loop. Unlike the `break` statement, it does not terminate the entire loop. Only a specific iteration of the loop is skipped, and the remaining code of that iteration is not executed after the `continue` statement is encountered, allowing the loop to start a new iteration.

Nested Loop Case

When a `continue` statement is encountered within the body of an inner loop, it terminates the execution of the current iteration of that inner loop and moves on to the next iteration of the same inner loop. Importantly, the execution of the outer loop remains unaffected.

Syntax

```
continue;
```

Structure of continue Statement in Looping Statements

1. for Loop

```
for (initialization; condition; increment/decrement)
{
    -----
    if (condition)
        continue;
    -----
}
next_statement;
```

2. while Loop

```
while (condition)
{
    -----
    if (condition)
```

```
        continue;
        -----
    }
    next_statement;
```

3. do-while Loop

```
do
{
    -----
    if (condition)
        continue;
    -----
}
while (condition);
next_statement;
```

Example: Continue Statement

```
public class ContinueExample {
    public static void main(String[] args) {
        for (int i = 1; i <= 5; i++) {
            if (i == 3) {
                continue; // Skip the rest of this iteration when i is 3
            }
            System.out.println(i);
        }
        System.out.println("Loop ended.");
    }
}
```

Explanation: - The for loop starts with `i = 1` and runs until `i <= 5`. - Inside the loop, there is an if statement that checks if `i` is equal to 3. - When `i` is 3, the `continue` statement is executed. This skips the rest of the code in that iteration, so `System.out.println(i)` is not executed for `i = 3`. - The loop then moves to the next iteration (`i = 4`) and continues normally. - For all other values (1, 2, 4, 5), the loop prints the value of `i`. - After the loop ends, the statement `System.out.println("Loop ended.");` is executed.

Output:

```
1
2
4
```

5
Loop ended.